



UNIVERSITY GREEN CHALLENGE

Project Report

A desktop sustainability engagement application that rewards verified eco-actions on campus.

Prepared for Project Assessment

August 2026

Scope: This report documents the implemented Python application, its architecture, testing evidence, SDG alignment, constraints, and opportunities for future development.

Executive Summary

University Green Challenge is a local desktop application designed to make everyday sustainable choices visible, reviewable, and rewarding. Students register, submit photo-supported eco-action claims, monitor their progress, and view a leaderboard. Administrators review the queue, update statuses, approve or reject claims, and inspect analytics. The application uses CustomTkinter for the interface and a small Python domain layer for validation, scoring, persistence, and evidence reuse prevention.

Key result: The implemented core rule checks passed 5 out of 5 isolated tests: registration, claim submission, duplicate-evidence rejection, approval scoring, and rejection trust-score handling.

Report Structure

Section	Content
2.1	Introduction and SDG Background
2.2	Problem Statement
2.3	System Objectives
2.4	Application Design and UML Class Diagram
2.5	Implementation Details
2.6	Testing and Sample Outputs
2.7	Discussion and Limitations
2.8	Conclusion

Evidence Base

The application-specific claims in this report are derived from the current project source code and saved demonstration data. SDG context is supported by the official United Nations Sustainable Development Goals pages listed in References.

2.1. Introduction and SDG Background

Universities are practical settings for sustainability action because students make repeated choices about transport, waste, energy, and consumption. However, informal activities are often difficult to record, verify, and recognize. University Green Challenge converts selected actions into a simple engagement cycle: action, evidence, review, points, feedback, and public ranking.

The project is directly aligned with SDG 12, Responsible Consumption and Production, by encouraging recycling, reusable bottles, and reduced waste. It also supports SDG 13, Climate Action, through lower-carbon mobility and energy-saving behaviours, and SDG 11, Sustainable Cities and Communities, through walking, cycling, and public transport. These connections are behavioural rather than measured emissions reductions; the application records participation and incentives, not verified carbon accounting. [1][2][3]

SDG	Application link	Examples implemented
SDG 12	Responsible use of materials and waste reduction.	Recycle campus waste; use a reusable bottle.
SDG 13	Encourage actions that can reduce operational emissions.	Turn off unused devices; choose active mobility.
SDG 11	Support sustainable and accessible transport choices.	Walk/cycle to class; use public transport.

Table 1. SDG alignment of the implemented eco-actions. Sources: official UN SDG pages [1][2][3].

2.2. Problem Statement

Campus sustainability initiatives can struggle to maintain student participation when there is no immediate feedback, transparent recognition, or consistent way to review evidence. Manual records are fragmented and a purely trust-based approach may allow duplicate or weak submissions. Administrators also need a manageable queue rather than informal messages or spreadsheets.

The project addresses five practical gaps:

- Students need a quick way to record a completed eco-action and see its status.
- Administrators need a controlled review process that can approve, reject, or request additional evidence.
- The challenge needs a visible, stable leaderboard to reinforce participation.
- Evidence should not be reused under a different filename.
- The application state should persist between local sessions without requiring a database server.

2.3. System Objectives

Objective	Implemented mechanism	Success indicator
Register participants	Name and student ID validation; duplicate-ID prevention.	New unique student can be stored.
Capture eco-actions	Six selectable actions with predefined point values and image evidence.	Claim enters pending review.
Review fairly	Admin queue, statuses, notes, approval/rejection, trust-score changes.	Every pending claim has an auditable final status.
Reward participation	Points and deterministic leaderboard sorting.	Approved claims update the ranking.
Protect evidence integrity	SHA-256 fingerprint comparison of image bytes.	Reused evidence is rejected.
Retain data locally	JSON load/save after successful state changes.	Records reappear after restart.

Table 2. Objectives traced to concrete functions in the implementation.

2.4. Application Design

The application follows a lightweight layered design. The interface controls screen changes and user input; the store applies rules; dataclasses define the domain model; and the persistence module serializes the current state to JSON. This separation keeps the most important rules testable without loading the graphical interface.

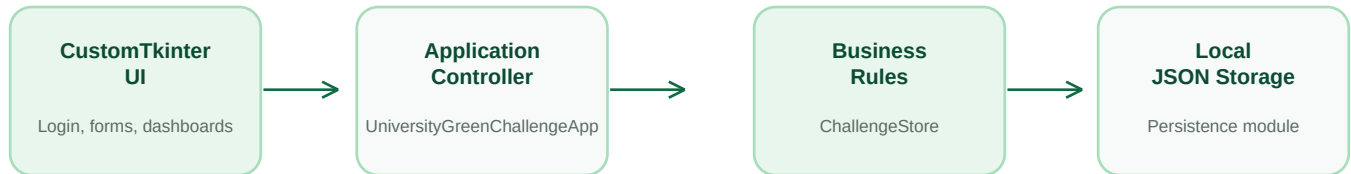
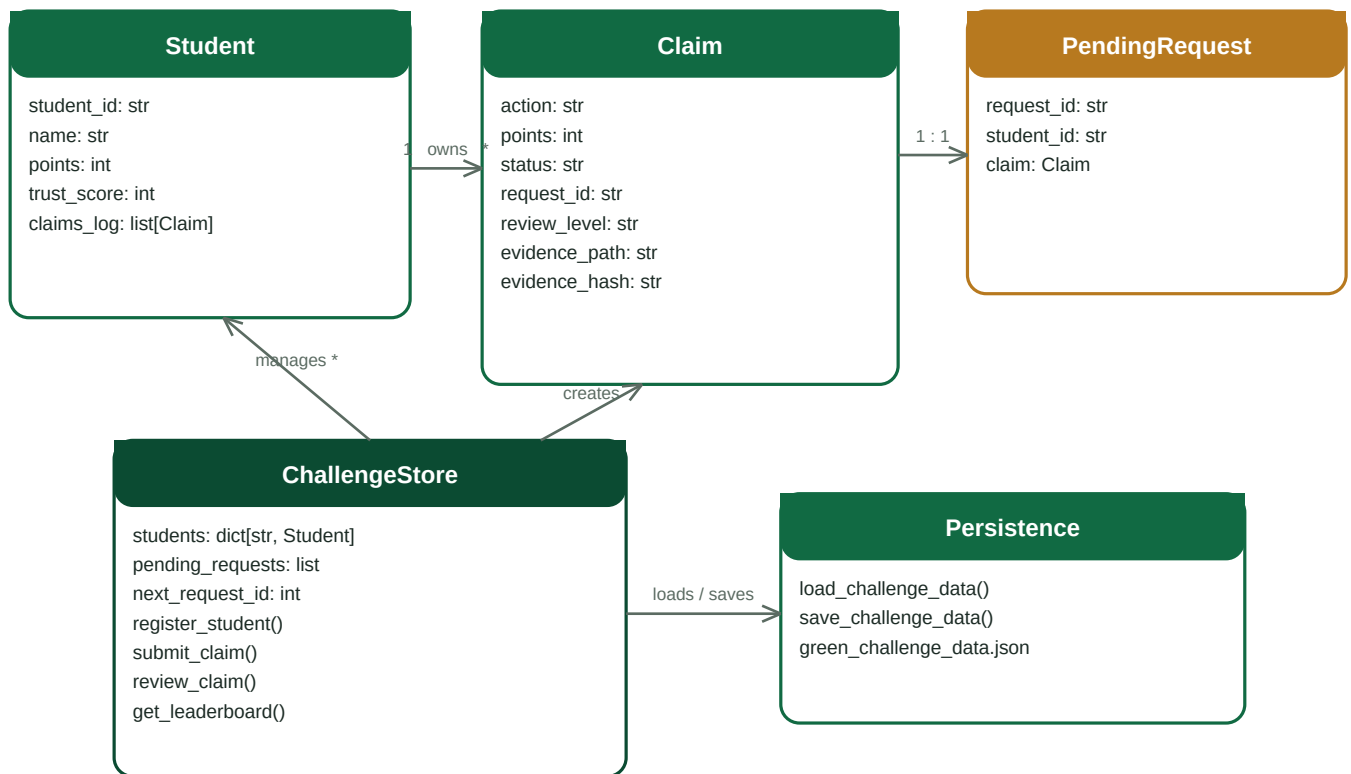


Figure 1. Application architecture.

UML Class Diagram



UML relationships reflect the implemented data classes and ChallengeStore operations.

Figure 2. UML class diagram for the implemented domain model.

Relationship summary: a **Student** owns a claim history; a **Claim** stores action, point value, status, evidence metadata, and review metadata; a **PendingRequest** references a claim awaiting a decision; and **ChallengeStore** coordinates registration, review, ranking, and persistence.

2.5. Implementation Details

Technology Stack

Layer	Implementation	Role
Entry point	university_green_challenge.py	Launches the desktop application.
User interface	CustomTkinter and tkinter widgets	Login, registration, forms, tables, dashboards, dialogs.
Domain model	dataclasses	Student, Claim, and PendingRequest structures.
Business logic	ChallengeStore	Validation, scoring, trust changes, queue handling, leaderboard sorting.
Persistence	JSON file	Local state stored in green_challenge_data.json.
Evidence integrity	hashlib.sha256	Blocks re-use of an image by comparing file-content fingerprints.

Table 3. Main implementation components.

Business Rules

- The system normalizes student names and IDs, then rejects empty or duplicate IDs.
- A submitted action must be one of six configured eco-actions. Their values are 5, 8, 10, 12, 15, or 25 points.
- The evidence image is read in blocks and hashed with SHA-256. A matching historical hash raises an error before a new claim is saved.
- Approval adds the claim points and increases trust by up to 5, capped at 100. Rejection reduces trust by 20, floored at 0.
- Leaderboard order is stable: points descending, trust score descending, name ascending, then student ID ascending.

Configured Eco-Actions

Eco-action	Points
Recycled campus waste	10
Walked or cycled to class	15
Used a reusable bottle	5
Turned off unused devices	8
Joined a campus clean-up	25
Used public transport to campus	12

Table 4. Point configuration from constants.py.

2.6. Testing and Sample Outputs

Testing was performed on 24 August 2026 using an isolated temporary data file and unique evidence files. This avoids altering the saved demonstration data. The test covers business rules directly through ChallengeStore, which is appropriate because the store contains the validation and scoring logic used by the interface.

Test case	Expected result	Observed result
Register a unique student	Student is stored with the supplied ID.	PASS - T001 registered.
Submit a valid claim	Claim is pending with configured points.	PASS - reusable bottle claim stored with 5 points.
Reuse the same image	Second claim is rejected before save.	PASS - ValueError raised for duplicate evidence.
Approve a claim	Points update; status becomes Admin-Approved.	PASS - score updated to 5; trust remained 100.
Reject a claim	Status becomes Rejected; trust decreases by 20.	PASS - trust changed from 100 to 80.

Table 5. Core business-rule test results: 5/5 passed.

Sample test output

```
Saved demo dataset: 7 students, 22 claims, 18 approved, 4 rejected Core rule tests: registration=PASS, claim submission=PASS, duplicate evidence rejected=PASS, approval updates score=PASS, rejection lowers trust=PASS  
Result: 5 / 5 passed
```

Current Saved Demonstration Data

Metric	Value
Registered students	7
Stored claims	22
Approved claims	18
Rejected claims	4
Points currently awarded	270
Most frequent recorded action	Joined a campus clean-up

Table 6. Summary computed from the current green_challenge_data.json file.

2.7. Discussion and Limitations

The main design strength is the direct connection between behaviour, review, and feedback. Students receive a clear outcome rather than an informal acknowledgement, while administrators retain control over point allocation. Separating ChallengeStore from the user interface also makes the core logic easier to test and extend.

Area	Current limitation	Suggested next step
Security	The administrator password is a demo constant in source code.	Use hashed credentials and role-based authentication.
Storage	JSON is local and suitable only for small, single-machine use.	Move to SQLite or a managed database for concurrency and backups.
Evidence	Hashing prevents exact file reuse but does not judge whether a photo proves the action.	Add structured review rubrics, timestamps, or supervised verification.
Fairness	Extra-scrutiny selection is probabilistic and may feel inconsistent to users.	Use explicit, explainable criteria with a review audit trail.
Integration	No institutional sign-in, notification, or carbon-calculation integration.	Connect campus SSO, email alerts, and an approved impact methodology.
Accessibility	The report does not verify full keyboard or screen-reader coverage of the desktop UI.	Conduct accessibility testing with target users and document findings.

Table 8. Practical limitations and future improvements.

Important interpretation: points are a participation incentive, not a quantified measure of environmental impact. Any future claims about carbon savings should use a transparent methodology and verified activity data.

2.8. Conclusion

University Green Challenge demonstrates a focused, maintainable approach to campus sustainability engagement. It provides a complete local workflow from registration through evidence submission, administrative review, point allocation, trust adjustments, analytics, and ranking. The project supports sustainability learning through visible, repeatable actions that align with SDG 11, SDG 12, and SDG 13.

The tested rules confirm that the implemented core flow protects unique registration, captures claims, blocks duplicate evidence, and applies approval or rejection outcomes correctly. The strongest next development priorities are secure authentication, multi-user storage, more explainable review criteria, and institution-level integrations.

References

- [1] United Nations Department of Economic and Social Affairs. Goal 12: Responsible Consumption and Production. <https://sdgs.un.org/goals/goal12>
- [2] United Nations Department of Economic and Social Affairs. Goal 13: Climate Action. <https://sdgs.un.org/goals/goal13>
- [3] United Nations Department of Economic and Social Affairs. Goal 11: Sustainable Cities and Communities. <https://sdgs.un.org/goals/goal11>
- [4] University Green Challenge project source code: `green_challenge/app.py`, `store.py`, `models.py`, `persistence.py`, `constants.py`, and `university_green_challenge.py`.
- [5] University Green Challenge saved demonstration data: `green_challenge_data.json`, accessed 24 August 2026.